

Nama: Malvin Juliyyan Nugraha

Kelompok Onyx

Tugas: Fase 1 - Pemrograman Dasar TLS 2026

Problem 1 (The Last Astronaut)

Pseudocode

Plaintext

PROGRAM The_Last_Astronaut

DECLARE

N, K, sisa, posisi_sekarang, langkah, tereliminasi
aktif AS ARRAY OF BOOLEAN

BEGIN

INPUT N

INPUT K

FOR i FROM 1 UNTIL N

SET aktif[i] = TRUE

END FOR

SET sisa = N

SET posisi_sekarang = 1

WHILE sisa > 1

SET langkah = (K - 1) MOD sisa

WHILE langkah > 0

SET posisi_sekarang = posisi_sekarang + 1

IF posisi_sekarang > N THEN

```

    SET posisi_sekarang = 1
END IF

IF aktif[posisi_sekarang] == TRUE THEN
    SET langkah = langkah - 1
END IF
END WHILE

SET tereliminasi = posisi_sekarang
SET aktif[tereliminasi] = FALSE
PRINT "Astronot yang dieliminasi: ", tereliminasi

IF tereliminasi MOD 2 == 0 THEN
    SET K = K + 2
ELSE
    SET K = K - 1
END IF

IF K < 2 THEN
    SET K = 2
END IF

SET sisa = sisa - 1

IF sisa > 0 THEN
    WHILE aktif[posisi_sekarang] == FALSE
        SET posisi_sekarang = posisi_sekarang + 1
        IF posisi_sekarang > N THEN
            SET posisi_sekarang = 1

```

END IF

END WHILE

END IF

END WHILE

PRINT "Astronot terakhir yang bertahan: ", posisi_sekarang

END

Penjelasan Logika

Algoritma ini pada dasarnya menyimulasikan proses eliminasi astronot secara melingkar. Setiap astronot direpresentasikan sebagai elemen dalam array boolean bernama aktif, dengan nilai TRUE berarti masih bertahan dan FALSE berarti sudah di geser. Selama masih ada lebih dari satu orang yang tersisa, program akan terus berputar melakukan eliminasi yang dikendalikan oleh variabel sisa, yang berfungsi sebagai tanda kapan pengulangan WHILE harus berhenti, yaitu ketika hanya tinggal satu astronot yang bertahan.

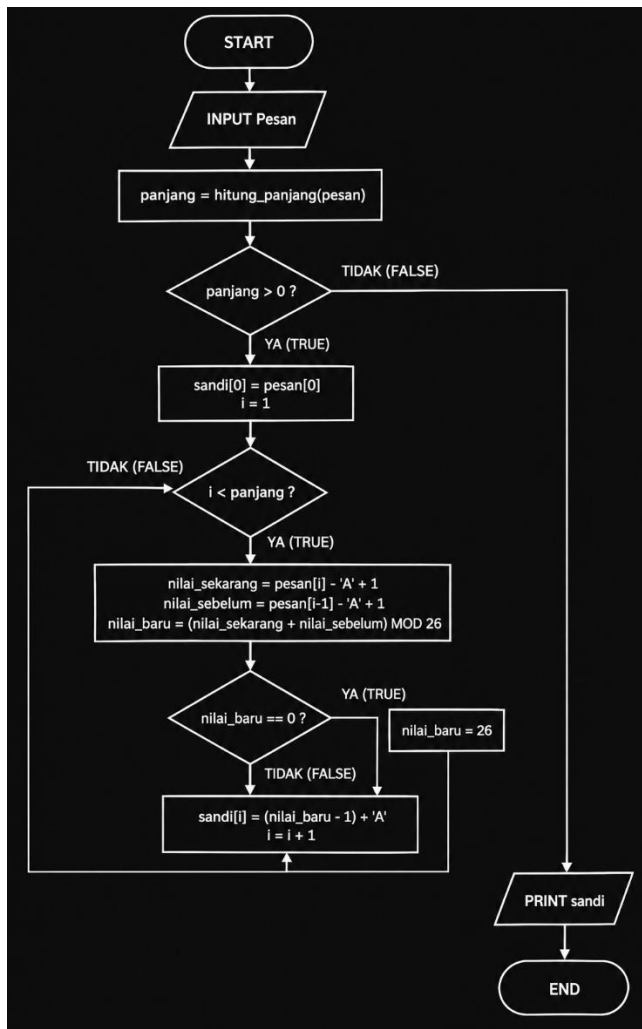
Perhitungan posisi siapa yang akan dieliminasi tidak dilakukan dengan cara memutar array satu per satu, namun memanfaatkan operasi modulo: $\text{langkah} = (K - 1) \text{ MOD } \text{sisa}$. Sehingga program tidak perlu berputar berkali-kali.

Setelah target ditemukan, astronot pada indeks itu langsung ditandai FALSE, sisa dikurangi satu, dan nomornya dicetak sebagai peserta yang gugur. Di fase ini, program ke blok IF-ELSE untuk menentukan nilai K yang baru, tergantung apakah nomor astronot yang baru saja tereliminasi itu genap atau ganjil. Supaya sistem tidak macet atau berjalan aneh saat K hasil perhitungan turun terlalu rendah, ada proteksi tambahan: jika K jatuh di bawah 2, nilainya langsung dipaksa jadi 2.

Terakhir, sebelum masuk ke kode berikutnya, ada pengulangan WHILE kecil yang tugasnya menggeser pointer posisi_sekarang ke indeks terdekat berikutnya yang masih bernilai TRUE. Ini memastikan hitungan eliminasi ke selanjutnya dimulai tepat dari astronot aktif yang berada persis di sebelahnya.

Problem 2 (Alien-In-The-Middle)

Flowchart



Penjelasan Logika

Algoritma ini bekerja dengan memproses string karakter secara urut. Langkah awalnya adalah menghitung total panjang pesan masukan, yang nantinya dipakai sebagai batas perulangan. karakter pertama (indeks ke-0) langsung disalin ke array hasil sandi, tanpa perlu diproses lebih lanjut.

Penyandian baru dimulai dari karakter kedua (indeks ke-1), lewat sebuah perulangan yang berjalan selama indeks masih berada dalam batas panjang pesan. Di setiap iterasi, huruf pada posisi saat ini terlebih dulu diubah menjadi angka $A = 1$ sampai $Z = 26$, lalu nilai tersebut dijumlahkan dengan nilai angka dari huruf asli di posisi sebelumnya.

Karena penjumlahan ini dapat melewati batas Z (26), algoritma menggunakan operator modulo 26 supaya perhitungan otomatis berbalik ke awal abjad. Namun kalau hasil penjumlahannya tepat kelipatan 26, modulo akan menghasilkan 0, nahh padahal 0 bukan representasi huruf yang valid. Untuk itu, ditambahkan pengecekan IF sederhana yang

mengubah nilai 0 tersebut menjadi 26, sehingga tetap terbaca sebagai huruf Z. Setelah nilai akhirnya valid, angka tersebut dikonversi kembali ke huruf, dan proses ini berulang sampai seluruh pesan selesai di encode.